

“高斯消去”编程实验/期末报告选题介绍

1、实验内容安排

平时并行编程作业：普通高斯消去

期末研究报告新增内容：特殊高斯消去——Gröbner 基的计算

2、普通高斯消去介绍

2.1 问题描述

普通高斯消去（LU 分解）计算的串行算法如下面伪代码所示：

```
1. procedure LU (A)
2. begin
3.   for  $k := 1$  to  $n$  do
4.     for  $j := k+1$  to  $n$  do // 第  $k$  行对角线元素变为 1
5.        $A[k, j] := A[k, j]/A[k, k]$ ;
6.     endfor;
7.      $A[k, k] := 1.0$ ;
8.     for  $i := k+1$  to  $n$  do
9.       for  $j := k+1$  to  $n$  do // 从第  $i$  行消去第  $k$  行
10.         $A[i, j] := A[i, j] - A[i, k] \times A[k, j]$ ;
11.      endfor;
12.       $A[i, k] := 0$ ;
13.    endfor;
14.  endfor;
15. end LU
```

计算模式如图 1 所示（第 k 步时第 k 行的除法运算，和后续行减去第 k 行的运算），系数矩阵只有阴影部分还参与计算，空白部分已不再参与计算。

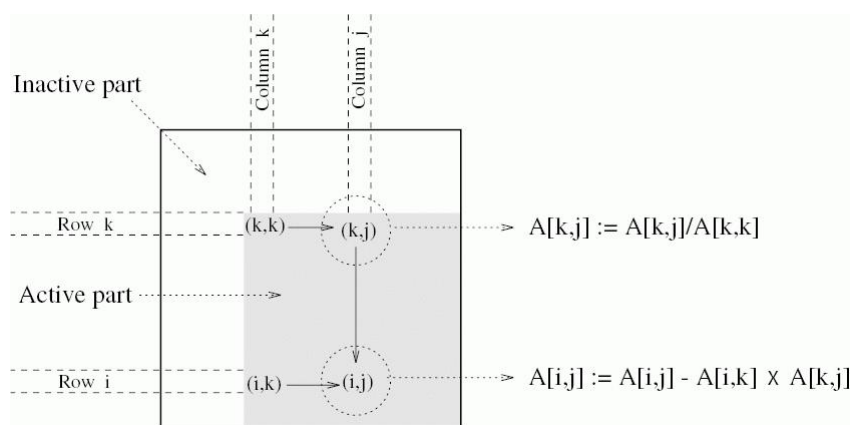


图 1. 高斯消去法计算示意图

算法过程示例:

$$\begin{bmatrix} 2 & 4 & 2 & 6 \\ 4 & 9 & 1 & 15 \\ 2 & 1 & 11 & 13 \\ 6 & 15 & 13 & 43 \end{bmatrix}$$

原始矩阵

$$\begin{bmatrix} \textcolor{red}{1} & 2 & 1 & 3 \\ \textcolor{blue}{0} & 1 & -3 & 3 \\ \textcolor{blue}{0} & -3 & 9 & 7 \\ \textcolor{blue}{0} & 3 & 7 & 25 \end{bmatrix}$$

k=1

$$\begin{bmatrix} 1 & \textcolor{red}{1} & 2 & 3 \\ 0 & \textcolor{red}{1} & -3 & 3 \\ 0 & \textcolor{blue}{0} & 0 & 16 \\ 0 & \textcolor{blue}{0} & 16 & 16 \end{bmatrix}$$

k=2

$$\begin{bmatrix} 1 & 2 & \textcolor{red}{1} & 3 \\ 0 & 1 & \textcolor{red}{-3} & 3 \\ 0 & 0 & \textcolor{red}{1} & 1 \\ 0 & 0 & \textcolor{blue}{0} & 16 \end{bmatrix}$$

k=3

$$\begin{bmatrix} 1 & 2 & 1 & 3 \\ 0 & 1 & -3 & 3 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

k=4

2.2 并行化思路

见后续各次实验教学指导书。

3 Gröbner 基计算中的高斯消去介绍

3.1 问题描述及数据集描述

特殊高斯消去计算来自一个实际的密码学问题——Gröbner 基的计算，与普通高斯消去计算的区别如下：

1. 运算均为有限域 GF(2)上的运算，即矩阵元素的值只可能是 0 或 1。其加法运算实际上为异或运算： $0+0=0$ 、 $0+1=1$ 、 $1+0=1$ 、 $1+1=0$ 。由于异或运算的逆运算为自身，因此减法也是异或运算。乘法运算 $0*0=0$ 、 $0*1=0$ 、 $1*0=0$ 、 $1*1=1$ 。因此，高斯消去过程中实际上只有异或运算——从一行中消去另一行的运算退化为减法。
2. 矩阵行分为两类，“消元子”和“被消元行”，在输入时即给定。消元子是在消去过程中充当“减数”的行，不会充当“被减数”。所有消元子的首个非零元素（即首个 1，称为首项）的位置（可通过将消元子放置在特定行来令该元素位于矩阵对角线上）都不同，但不会涵盖所有对角线元素。被消元行在消去过程中充当“被减数”，但有可能恰好包含消元子中缺失的对角线 1 元素，此时它“升格”为消元子，补上此缺失的对角线 1 元素。
3. 此特殊的高斯消去计算过程（串行算法）如下：
 - a) 实际问题中矩阵规模很大，消元子和被消元行的数量很多（可能达到百万级），大大超出内存容量，一种处理方式是逐批次将消元子和被消元行读入内存，执行下面步骤 b) -c)；
 - b) 对当前批次中每个被消元行，检查其首项，如有对应消元子，则将其减去（异或）对应消元子，重复此过程直至其变为空行（全 0 向量）或首项不在当前批次覆盖范围内、或首项在范围内但无对应消元子；
 - c) 如果某个被消元行变为空行，则将其丢弃，不再参与后续消去计算；如其首项被当前批次覆盖，但没有对应消元子，则将它“升格”为消元子，在后续消去计算中将以消元子身份而不再以被消元行的身份参与；
 - d) 重复上述过程，直至所有批次都处理完毕，此时消元子和被消元行共同组成结果矩阵——可能存在很多空行。
4. 说明：
 - a) 批次：小心组织数据读入、计算次序，保证所有消元子和被消元行都两两处理过，可考虑不同批次划分和计算次序。对我们的编程实验，若矩阵规模不大，可全部读入内存，不分批次。
 - b) 如按稠密矩阵存储，可采用位存储方式，即，用整型值的一个比特保存矩阵的一个元素。这样，一对整数的异或运算即可完成很长的子向量的异或。SIMD 并行化也很方便。当然，矩阵很稀疏，也可采用稀疏矩阵存储方式，SIMD 并行化相对复杂。
5. 输入数据格式：每个测试用例通过一个文本文件提供，先给出消元子，然后给出被消元行。每个消元子/被消元行占据一行文本，用稀疏向量方式表示——用整数表示 1 元素所在列。例如，矩阵列数为 10 列，某行内容为 9 3 1，则表示行向量 1 0 0 0 0

0 1 0 1 0 (最左为列 9, 最右为列 0)。

特殊高斯消去的一个示例:

共 10 个消元子, 矩阵列数 26

[illegible]

6 个被消元行

[illegible]

4	0	1	0	1	0	0	0	1	0	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0
5	1	0	0	0	0	1	0	0	0	1	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0

假设每轮处理 5 个消元子，第二轮读入消元子：

首项 列号	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
20	NULL																										
19	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0	
18	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0	
17	NULL																										
16	NULL																										

6 个被消元行

行号 列号	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	1	0	0	0	1	0	1	0	1	1	0	0	1	0	0	1	0	0	0
1	0	0	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	0
2	0	0	0	0	0	0	1	0	0	1	1	1	0	0	0	0	1	0	0	1	1	0	0	0	0	0
3	0	0	0	0	0	0	1	0	0	1	1	0	1	0	0	0	1	0	0	1	0	0	0	1	0	0
4	0	0	0	0	0	0	0	1	1	0	0	0	1	0	0	1	0	1	0	1	0	0	0	0	0	0
5	0	0	0	0	0	1	0	1	0	1	0	1	1	0	0	0	0	0	0	1	1	0	1	0	0	0

被消元行 0 减去消元子 18，得到

行号 列号	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	1	0	1	0	1	0	1	1	1	1	0	0	0	1	0	0	0

因为不存在首项为 16 的消元子，因此该行“升格”为消元子。类似的，被消元行 1 首项为 17、被消元行 5 首项为 20，也“升格”为消元子。此批次消元子变为：

首项 列号	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
20	0	0	0	0	0	1	0	1	0	1	0	1	1	0	0	0	0	0	0	1	1	0	1	0	0	0
19	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0
18	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0
17	0	0	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	0
16	0	0	0	0	0	0	0	0	0	1	0	1	0	1	0	1	1	1	1	0	0	0	1	0	0	0

对被消元行 2-4 继续进行消去计算，得到：

行号	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	---	---	---	---	---	---	---	---	---	---

并行，此时就需要结合 3)、2) 的消元顺序来设计适合的并行任务划分，在提高并发度的同时保证正确性。

- b) 对类倒排链表存储方式，可自行探索。
- c) 当矩阵规模非常庞大，需使用外存算法时，并行算法的设计就要同时考虑计算和访存。例如，多线程并行化时可考虑计算和访存异步模式，在前台线程进行消元计算时、后台线程读取下一步要处理的数据，此时就要仔细设计计算和 I/O 步骤，降低依赖关系，以便实现异步模式。